

Create Database Connection

Overview

The database connection establishes communication between the DarshanEase backend application and the MongoDB database. The application uses **Mongoose**, an Object Data Modeling (ODM) library, to connect to MongoDB and perform database operations efficiently.

Step 1: Install Mongoose

Install Mongoose in the backend project.

```
npm install mongoose
```

Step 2: Configure Environment Variables

Create a **.env** file in the backend folder and store the MongoDB connection string.

For Local MongoDB

```
PORT=5000
MONGODB_URI=mongodb://localhost:27017/darshanease
JWT_SECRET=your_secret_key
```

For MongoDB Atlas

```
PORT=5000
MONGODB_URI=mongodb+srv://username:password@cluster0.xxxxx.mongodb.net/
darshanease
JWT_SECRET=your_secret_key
```

Step 3: Create Database Configuration File

Create a new file:

```
backend/config/db.js
```

Add the following code:

```
const mongoose = require("mongoose");

const connectDB = async () => {
  try {
    await mongoose.connect(process.env.MONGODB_URI);
```

```

    console.log("✅ MongoDB Connected Successfully");
  } catch (error) {
    console.error("❌ Database Connection Failed");
    console.error(error.message);
    process.exit(1);
  }
};

```

module.exports = connectDB;

Explanation

- mongoose.connect() establishes a connection with MongoDB.
- process.env.MONGODB_URI reads the database URL from the .env file.
- try...catch handles connection errors gracefully.
- process.exit(1) stops the server if the database connection fails.

Step 4: Configure the Main Server File

Open **server.js** and import the database connection.

```

require("dotenv").config();

const express = require("express");
const cors = require("cors");
const connectDB = require("../config/db");

const app = express();

// Connect to MongoDB
connectDB();

// Middleware
app.use(cors());
app.use(express.json());

// Test Route
app.get("/", (req, res) => {
  res.send("DarshanEase Server Running...");
});

// Start Server

```

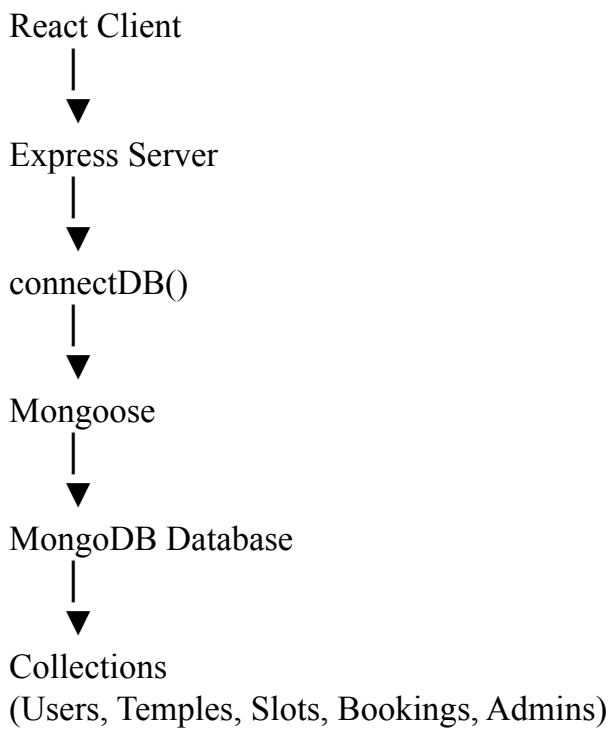
```
const PORT = process.env.PORT || 5000;

app.listen(PORT, () => {
  console.log(`Server is running on port ${PORT}`);
});
```

Project Structure

```
backend/
├── config/
│   └── db.js
├── controllers/
├── middleware/
├── models/
├── routes/
├── .env
├── server.js
└── package.json
```

Database Connection Flow



Testing the Connection

Run the backend server:

```
npm run dev
```

Expected output:

✅ MongoDB Connected Successfully

Server is running on port 5000

If an error occurs, verify:

- MongoDB is running (for local installation).
- The MONGODB_URI in the .env file is correct.
- Internet connectivity (for MongoDB Atlas).
- Database user credentials and IP whitelist (for Atlas).

Benefits of Using Mongoose

- Simplifies MongoDB operations.
- Supports schema-based data modeling.
- Provides built-in validation.
- Handles relationships between collections.
- Improves code readability and maintainability.

Conclusion

The database connection is a fundamental component of the DarshanEase backend. By using Mongoose with environment-based configuration, the application securely connects to MongoDB, enabling efficient storage and retrieval of data such as users, temples, darshan slots, and bookings while following best practices for scalability and maintainability.